Univerza v Ljubljani
Fakulteta za računalništvo in informatiko

Žan Hribar

Program za analiziranje šahovskih pozicij

diplomsko delo

visokošolski strokovni študijski program
prve stopnje
računalništvo in informatika

Mentor: izr. prof. dr. Aleksander Sadikov

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani `http://www.gnu.org/licenses/`.

Besedilo je oblikovano z urejevalnikom besedil LaTeX.

Kandidat: Žan Hribar
Naslov: Program za analiziranje šahovskih pozicij
Vrsta naloge: Diplomska naloga na visokošolskem strokovnem študijskem programu prve stopnje Računalništvo in informatika
Mentor: izr. prof. dr. Aleksander Sadikov

Opis:
Študent naj zasnuje šahovski program za igranje in analiziranje pozicij. Primarni namen naloge je ugotoviti kako dober program je možno spisati brez zunanje pomoči v okvirnem roku treh mesecev. Seveda se naj pri tem študent opira na obstoječo literaturo, vendar pa ne razširja obstoječih šahovskih programov.

Title: A chess analysis program

Description:
The student should design a chess program for playing and analysing positions. The primary purpose of the assignment is to determine how good a program can be written without external help within roughly three months. Of course, the student should rely on existing literature, but not expand on existing chess programs.

Zahvaljujem se mentorju za sprejem tako specifične teme, strokovno usmerjanje in pomoč pri testiranju končnega izdelka.

Posebna zahvala gre tudi moji družini, prijateljem in kolegom za vso podporo, spodbudo in motivacijo skozi celoten proces izdelave naloge.

Kazalo

Povzetek

Abstract

1 Uvod **1**

2 Sorodna dela **3**

3 Predstavitev figur in pozicij **5**

4 Predstavitev potez **9**

4.1 Generiranje pseudo legalnih potez 9

4.2 Predstavitev strukture potez 9

4.3 Generiranje potez z bitnimi tabelami 10

4.4 Generiranje potez z magičnimi bitnimi tabelami 12

4.5 Generiranje posebnih potez 13

4.6 Testiranje pravilnosti generiranih potez 14

4.7 Igranje potez 14

4.8 Odigraj potezo 15

4.9 Izniči potezo 16

4.10 Testiranje pravilnosti igranja potez 17

5 Iskanje najboljše poteze **19**

5.1 Iskanje po vseh možnih potezah 20

5.2 Preverjanje legalnosti poteze 22

5.3 Iterativno poglabljanje 23
5.4 Iskanje quiescence 23
5.5 Preverjanje trikratne ponovitve pozicije 25

6 Ocena pozicije **27**
6.1 Ocena materiala 27
6.2 Ocena mobilnosti 28
6.3 Ocena pozicije figur 28
6.4 Ocena varnosti 28

7 Iskalne optimizacije **31**
7.1 Transpozicijske tabele 31
7.2 Urejanje potez 34
7.3 Rezanje null move 36
7.4 Iskanje PVS 37
7.5 LMR 37
7.6 Rezanje mate distance 38

8 Ocenjevanje in rezultati **39**
8.1 Vmesnik za komunikacijo 39
8.2 Načini ocenjevanja in rezultati 40

9 Zaključek **45**

Literatura **47**

Povzetek

Naslov: Program za analiziranje šahovskih pozicij

Avtor: Žan Hribar

Predstavili bomo potek izdelave programa za analiziranje šahovskih pozicij. V zadnjih nekaj desetletjih se je nivo šahovske teorije in taktike močno zvišal. Velik razlog za ta napredek so programi za analiziranje šahovskih pozicij. Predstavili bomo različne predstavitve figur in pozicij v računalniškem šahu ter kaj uporabljajo moderni programi. Nato se bomo obrnili na način generiranja potez. Nadaljevali bomo z prikazom načina igranja in izničevanja potez. Obdelali bomo različne iskalne algoritme in njihovo uporabo. Predstavili bomo različne hevristike za ocenjevanje pozicij. Obrnili se bomo tudi na nekatere iskalne optimizacije, ki pohitrijo delovanje programa. Pokazali bomo tudi vmesnik komunikacije, ki ga uporabljajo programi za analiziranje šahovskih pozicij za medsebojno komunikacijo. Prikazali bomo rezultate testov, ki prikazujejo, da je naš program primerljiv s programom Stockfish 18 omejen na 2300 moči.

Ključne besede: Šah, Program za analiziranje šahovskih pozicij, Negamax, Hevristike, Iskalne optimizacije.

Abstract

Title: A chess analysis program

Author: Žan Hribar

We will present the development process of a chess engine. Over the last few decades, the level of chess theory and tactics has increased significantly. A major reason for this progress are chess engines. We will present different representations of pieces and positions in computer chess and what modern programs use. We will then turn to the method of move generation. We will continue by presenting the methods of making and unmaking moves. We will discuss various search algorithms and their use. We will present different heuristics for evaluating positions. We will also discuss some search optimizations that speed up the program. We will also show the communication interface used by chess engines and other programs for communication with one another. We will present the results of tests showing that our program is comparable to Stockfish 18 limited to a playing strength of 2300.

Keywords: Chess, Chess engine, Negamax, Heuristics, Search optimizations.

Poglavje 1

Uvod

Šah je že skoraj 1500 let stara igra. Čez vsa ta leta sta se stil in taktika igre zelo spremenila in izboljšala. Eden izmed večjih preskokov pri razumevanju in teoriji šaha se je zgodil v zadnjih nekaj desetletjih. Razlog za napredek je uvedba računalniških programov za analiziranje šahovskih pozicij. Čeprav se zdijo takšni programi dokaj nova tehnologija, so ljudje že na začetku 20. stoletja testirali ideje igranja šaha proti napravam [13]. Danes velika večina šahovske teorije temelji na takšnih programih, saj so mnogo boljši od človeških igralcev šaha. Zato je naš cilj poskusiti in predstaviti proces izdelave programa za analizo šahovskih pozicij z močjo okoli 2000 ratinga. V nadaljevanju bomo predstavili izdelavo vseh delov, ki jih potrebujemo za implementacijo programa za analiziranje šahovskih pozicij z modernimi tehnikami in optimizacijami. V poglavju 2 bomo predstavili potek razvoja programov za analiziranje šahovskih pozicij skozi zgodovino. V poglavju 3 bomo začeli s predstavitvijo figur in pozicij ter možnih alternativ. Po izbiri predstavitve bomo v 4. poglavju pokazali način generiranja potez. Nato bomo v istem poglavju prikazali način implementacije igranja in izničevanja potez. Sledili bo poglavje 6 z algoritmi iskanja najboljše poteze. Po implementaciji iskalnih algoritmov se bomo osredotočili na poglavje 6, kjer bomo predstavili hevristike ocenjevanja pozicije. Na tej točki bomo imeli popolnoma delujoč programa za analiziranje šahovskih pozicij. Ker želimo še povečati moč

programa, bomo predstavili poglavje 7 o raznih iskalnih optimizacijah, ki pripomorejo k hitrejšemu iskanju najboljše poteze. V zadnjem poglavju 8 bomo predstavili tudi način za medsebojno komunikacijo programov za analiziranje šahovskih pozicij. V zadnjem poglavju 8 bomo tudi predstavili rezultate in moč našega programa v primerjavi s programom Stockfish 18 [11] na različnih nivojih moči.

Poglavje 2

Sorodna dela

Prva znana naprava za igranje šaha je bila v 18. stoletju, in sicer naprava Mechanical Turk [12]. Naprava je bila v obliki človeka in je zmagovala proti večini igralcev. Pozneje je bilo razkrito, da je napravo upravljal mojster šaha iz tistega časa.

Prva resna naprava za igranje šaha se je pojavila leta 1912. Leonardo Torres Quevedo je izdelal prvo napravo, poimenovano El Ajedrecista [13]. El Ajedrecista je lahko zmagala končnico kralj in trdnjava proti kralju. Imela je celo možnost zaznavanja napačnih potez.

Naslednji korak v računalniškem šahu je bil v šestdesetih letih, ko sta Alan Turing in Claude Shannon predstavila enega izmed prvih algoritmov. Algoritem ni bil izveden na računalniku, saj so bili računalniki premalo zmogljivi, da bi algoritem lahko testirali. Dietrich Prinz je leta 1951 predstavil svoj šahovski algoritem, ki je lahko zmagal pozicije, kjer je bil prisoten mat v dveh potezah. Prvi program, ki je bil zmožen odigrati celotno igro, je bil izdelan leta 1957. Izdelal ga je inženir iz podjetja IBM Alex Bernstein [14].

Naslednji velik napredek sta bila odkritje in dokaz algoritma minimax z alfa beta rezanjem [2], ki ga uporabljajo programi za analizo šahovskih pozicij še danes. Razviti so bili tudi algoritmi za izbiranje potez, razne nove hevristike, iterativno poglabljanje ter implementacijo otvoritev in končnic. V zelo veliko pomoč je bila tudi hitrost razvijanja močnejše računalniške opreme,

kar je bila težava v času Turinga. Leta 1978 je bil predstavljen program Belle [10], ki je lahko preveril 30 milijonov pozicij v treh minutah in je postal prvi program, ki se je lahko primerjal z mojstri šaha.

Do leta 1996 se je področje šaha izboljševalo, ampak programi še vedno niso mogli premagati najboljših igralcev. Leta 1996 je takratni prvak v šahu Gary Kasparov igral 6 iger proti programu Deep Blue [5]. Čeprav je Kasparov zmagal 4 proti 2, se je prvič v zgodovini zgodilo, da je računalniški program premagal najboljšega človeškega igralca na svetu. Od takrat dalje so se programi tako izboljšali, da jih ljudje ne morejo več premagati.

Zadnji velik napredek v svetu računalniškega šaha je bil leta 2017, ko je bil izdelan prvi program za analiziranje šahovskih pozicij, ki je uporabljal nevronske mreže pri analizi. Ta program se imenuje AlphaZero [8]. Svoje učne podatke je pridobil tako, da je igral igre sam proti sebi. Nato so ta program testirali proti trenutnemu najboljšemu programu Stockfish [11]. Odigrala sta 100 iger, kjer je AlphaZero [8] zmagal 28 iger in remiziral preostalih 72. Kmalu po tem so še drugim programom dodali nevronske mreže v njihovo ocenjevalno funkcijo.

Poglavje 3

Predstavitev figur in pozicij

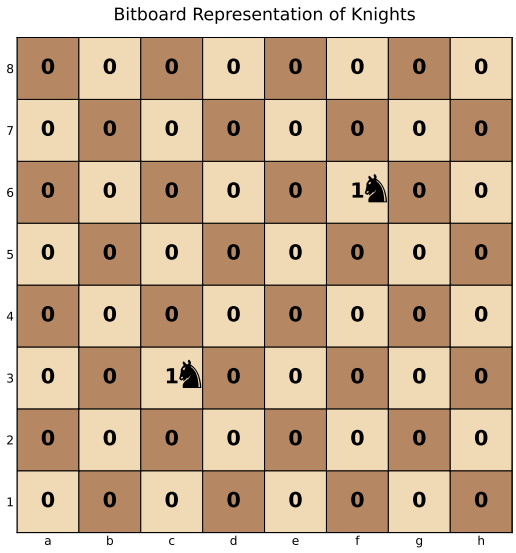
Najprej moramo pregledati, kakšni načini predstavitev figur in pozicij obstajajo in kateri bi bili najboljši za našo uporabo. Izbira predstavitve je pomembna, saj bo osnova za vse naslednje korake izdelave programa. V svetu računalniškega šaha so skozi zgodovino bili uporabljeni različni načini predstavitev pozicij in figur. Vsak izmed načinov ima svoje prednosti in svoje slabosti. Najbolj pogosto uporabljeni načini predstavitev so:

- **Seznami figur:** seznami figur so ena izmed najbolj preprostih predstavitev. Figure so shranjene v seznamih, ki so lahko razdeljeni na različne načine, kot so barva ali posebna urejenost znotraj seznama glede na figure. Figure znotraj seznama po navadi hranijo podatke o svoji poziciji. Prednost tega načina shranjevanja je, da izpusti prazna polja na šahovnici. Težava pa nastane pri generiranju potez, saj je pridobivanje informacij o povezanosti figur zelo potratno.

- **Seznam polj:** seznam polj uporablja dvodimenzionalen seznam 8x8 za predstavitev celotne šahovnice in figur. Vsako polje hrani informacijo o zasedenosti polja in katera figura je trenutno na določenem polju. Prednost tega načina shranjevanja je, da lahko zelo hitro ugotovimo, ali določena figura zaseda določeno mesto. Lažje je tudi ugotoviti povezanost med figurami. Slabost te predstavitve nastane pri generiranju

potez, ko iščemo figure, saj moramo pregledati celotno šahovnico, ki je v večini prazna, samo zato, da najdemo vse možne figure, ki se lahko premikajo.

- **Bitne tabele [2]:** bitne tabele shranjujejo podatke o figurah v 64 bitnih številkah, kjer vsak bit predstavlja eno polje na šahovnici. 1 v bitni tabeli pomeni zasedenost tistega polja, medtem ko vrednost 0 pomeni, da je polje prazno. Celotno šahovnico in vse figure lahko predstavimo z bitnimi tabelami tako, da shranimo eno bitno tabelo za vsak tip figure in jih razdelimo glede na barvo. Prednost tega načina shranjevanja je hitro generiranje in igranje potez, saj lahko z bitnimi operacijami nad različnimi bitnimi tabelami pridobimo podatke o zasedenosti polj in razmerjih med figurami. Največja prednost bitnih tabel je možnost predhodnega izračuna možnih premikov za vsako figuro na vsakem polju, kar zelo pospeši in olajša iskanje možnih potez. Slabost tega načina shranjevanja je kompleksnost implementacije. Predstavitev ni najbolj prijazna, vendar je zaradi učinkovitosti najpogosteje uporabljen način za shranjevanje v modernih programih za analizo šaha. Primer predstavitve si lahko ogledamo na sliki 3.1.



Slika 3.1: Predstavitev skakačev z bitnimi tabelami

Zaradi hitrosti naša implementacija programa za analizo šahovskih pozicij uporablja 15 bitnih tabel za predstav figur ter njihovih pozicij. Vsaka tip figure uporablja svojo bitno tabelo. Vsak tip je razdeljen še na barvo. Figure prinesejo 12 bitnih tabel. Glede pozicije so dodane še 3 bitne tabele za predstavitev črnih, belih in vseh figur. Poleg pozicijskih bitnih tabel implementacija uporablja še zgodovino potez, zgoščen izvleček pozicije, možno en passant polje in zastavice, ki vsebujejo informacije o rokadi obeh barv ter katera barva je trenutno na potezi.

Poglavje 4

Predstavitev potez

4.1 Generiranje pseudo legalnih potez

Naslednji korak po izbiri predstavitve pozicij in figur je način generiranja potez. Hitrost in pravilnost generiranja potez sta ključni lastnosti modernih programov za analizo šahovskih pozicij. To je funkcionalnost, ki se največkrat uporablja med analizo pozicije. Naša implementacija uporablja generiranje psevdo legalnih potez, kar pomeni, da generiramo vse možne premike figur glede na trenutno stanje pozicije, vendar generiranih potez ne preverjamo, ali bo odigrana poteza povzročila nelegalno pozicijo na šahovnici, kot je napaden kralj po premiku figure, saj bi bilo takšno preverjanje med generiranjem zelo potratno. Vseeno pa odstranimo poteze, kot so jemanje lastnih figur, saj je s pomočjo bitnih operacij nad bitnimi tabelami takšna operacija zelo hitra.

4.2 Predstavitev strukture potez

Moderni programi za analizo šaha po navadi uporabljajo zelo majhne strukture za shranjevanje možnih potez. Uporabljajo se 32 ali celo 16 bitne številke, v katerih so podatki določeni po bitih. Ti podatki so 6 bitov za polje, iz katerega se figura premakne, 6 bitov za polje, na katerega se premakne, preostalih 4 do 20 bitov pa se uporablja za različne zastavice, kot so promocije in katera figura

se je premaknila. Prednost take strukture je manjša poraba pomnilnika, kar posledično vpliva na hitrost izvajanja zaradi lokalnosti pomnilnika. Težava takšnega zapisa je dodana kompleksnost pri implementaciji igranja potez, zato smo se za našo implementacijo odločili uporabiti večjo strukturo, ki je človeku bolj prijazna za uporabo pri igranju in izničenju poteze. V strukturo shranjujemo bitne tabele za polje, iz katerega se figura premika, na katero polje se premika, prejšnje polje en passant za lažje izničenje poteze, zastavice in ocene poteze, ki se pozneje uporabljajo pri urejanju potez. V zastavicah pa imamo informacije o promocijski figuri, vzeti figuri, jemanju en passant, izgubi pravic rokade in katera figura je odigrala potezo.

4.3 Generiranje potez z bitnimi tabelami

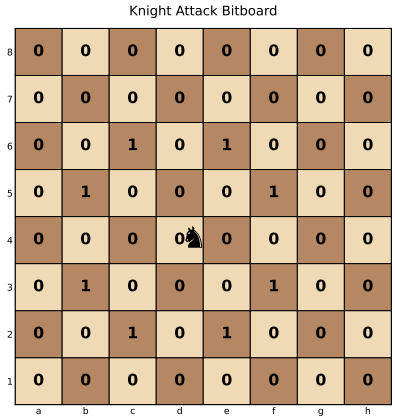
Kot že omenjeno, je velika prednost bitnih tabel možnost izračuna napadov vnaprej, kar nam omogoča hiter dostop do vseh možnih premikov za določeno figuro v konstantnem času. V ta namen se uporabljajo napadalne tabele. Napadalna tabela hrani masko oziroma bitno tabelo za vsako polje na šahovnici za specifično figuro. Ta maska nam pove, kam se figura iz določenega polja lahko premakne. Vrednost 1 v maski nam pove, da je premik na tisto določeno polje mogoč, vrednost 0 pa, da se figura ne more premakniti na tisto polje. Ker shranjujemo podatke o pozicij figur v bitnih tabelah, moramo najprej pozicijo figure pretvoriti v pravilni indeks polja. Ta operacija je trivialna, saj v modernih procesorjih obstaja operacija, ki prešteje število ničel do prve enke, kar pretvori našo bitno tabelo v pravilni indeks polja za napadalno tabelo. Nato lahko za vsako polje izračunamo možne premike in jih shranimo. Vse to izvedemo ob zagonu programa. Izračuni napadalnih tabel in generiranje potez pa se med figurami nekoliko razlikujejo. Načini shranjevanja napadalnih tabel in generiranja potez so:

- **Skakač:** pri računanju napadalnih tabel se uporablja maska, ki se generira glede na trenutno polje. Pri generiranju vsakega dela maske moramo biti pozorni, da se v masko doda samo v primeru, če je znotraj

šahovnice. Generiranje potez postane vpogled v tabelo. Nato sledita odstranitev potez, ki bi povzročile jemanje lastnih figur in pretvorba preostalih polj v bitni tabeli, označenih z 1, v prej definirano strukturo poteze. Primer maske si lahko pogledamo na sliki 4.1.

- **Kralj:** tako kot pri skakaču, se maska možnih premikov generira za vsako polje in se shrani v napadalno tabelo. Generiranje potez ostane podobno kot pri skakaču, z razliko pri rokadi, ki bo predstavljena v enem od naslednjih poglavij. Ker generiramo psevdo legalne poteze, se tukaj ne ukvarjamo s preverjanjem, ali bi odigrana kraljeva poteza povzročila nelegalno pozicijo.

- **Kmet:** kmet se od kralja in skakača pri generiranju napadalnih tabel nekoliko razlikuje, saj nima možnosti jemanja nazaj, kar pomeni, da kmeti potrebujejo dve različni napadalni tabeli glede na barvo. V bitnih napadalnih tabelah so shranjeni samo premiki, ki bi povzročili jemanje nasprotnikove figure. Pri generiranju potez pa ima kmet več posebnosti. Jemanje nasprotnkovih figur in premikanje pri kmetu nista enaka. Pri kmetu moramo biti pozorni na možnost premika za dve polji naprej in možnost promocije v primeru, da kmet pride do konca šahovnice, kjer je treba generirati vse možne promocije.



Slika 4.1: Predstavitev ene izmed mask napadalne tabele skakača

4.4 Generiranje potez z magičnimi bitnimi tabelami

Težava generiranja napadalnih tabel nastane pri figurah, kot so lovec, trdnjava in dama, saj ne moremo vnaprej izračunati maske možnih napadov, tako kot pri preostalih figurah, zaradi možnosti, da preostale figure ovirajo premik na določeno polje. Težava nastane tudi pri indeksiranju vseh možnih mask. Vseeno želimo imeti konstanten čas pri generiranju potez za te figure. V ta namen uporabljamo strukturo magičnih bitnih tabel [7]. Magične bitne tabele delujejo tako, da za določeno figuro na določenem polju izračunajo vse možne kombinacije figur, ki lahko blokirajo premik, nato dobljene maske pomnožijo z naključno generirano številko in vzamejo prvih n bitov dobljene številke za indeks in shranijo masko vseh možnih premikov v napadalno tabelo na indeksu polja in novega indeksa. V primeru, da na tistem indeksu že obstaja maska, ki je drugačna od nove, se napadalna tabela za tisto polje izprazni, izbrana je nova naključna številka in celoten proces se ponavlja, dokler ni odkrita številka, ki pri zgoščevanju ne povzroča kolizij različnih mask. Vsako polje ima svojo naključno številko, ki zadosti zgoraj navedenim pogojem. V naši implementaciji v primeru, da najdemo takšno naključno generirano številko za določeno polje, ki nam omogoča indeksiranje z n biti, le-to shranimo in poskušamo poiskati novo številko, ki omogoča indeksiranje z $n - 1$ biti. Ta proces ponavljamo, dokler po določenem številu poskusov ne najdemo primerne številke, ki bi zadostila pogojem. Nato shranimo indeks polja, število bitov in naključno generirano številko v datoteko csv. Imamo dve datoteki - ena z naključno generiranimi številkami za lovce in eno za trdnjave. Dama ne potrebuje svoje lastne datoteke številk, saj je njena maska možnih premikov sestavljen iz kombinacije premikov trdnjave in lovca. Ob zagonu programa se ti dve datoteki prebereta in uporabita pri indeksiranju vseh možnih mask premikov. Generiranje potez s pomočjo magičnih bitnih tabel:

- **Trdnjava, lovec:** naredi se vpogled v magično bitno tabelo za določeno

figuro v konstantnem času in se iz pridobljene maske premikov generirajo vse možne poteze.

- **Dama:** naredi se vpogled v magično bitno tabelo trdnjave in lovca ter se dobljene maske premikov z operacijo XOR združi v popolno masko daminih premikov. Iz te maske se nato generirajo možne poteze.

Za vpogled v tabelo potrebujemo masko vseh premikov na določenem polju za določeno figuro, postavitve vseh figur, naključno generirano številko in velikost indeksa. To si lahko pogledamo v izseku kode 1.

Izsek kode 1: Vpogled v napadalno tabelo trdnjav.

```
1 uint64_t getRookAttackPattern(int sqInd, uint64_t position, AttackTables
  ↪ *attackTables)
2 {
3     MagicTableHash magicTableHash =
      ↪ attackTables->rookMagicHashTable[sqInd];
4     uint64_t index = (position & magicTableHash.mask) *
      ↪ magicTableHash.magicNumber;
5     index = index >> magicTableHash.shift;
6
7     return attackTables->rookAttacks[sqInd][index];
8 }
```

4.5 Generiranje posebnih potez

Sedaj imamo implementacijo generiranja potez za vse figure. Manjka nam še generiranje posebnih potez, kot sta rokada in en passant. Generiranje teh potez postane bolj kompleksno, saj poleg trenutnega stanja pozicije in relacije med figurami potrebujemo še nekatere zgodovinske informacije o poteku igre. Generiranje posebnih potez:

- **Rokada:** pri generiranju teh potez naredimo izjemo pri pravilu o preverjanju legalnosti, saj vnaprej vemo, kakšna bo struktura poteze in potrebujemo samo informacijo, ali je ta poteza mogoča ali ne. Obstajajo

4 različne možne poteze - dolga in kratka rokada za belega in dolga ter kratka rokada za črnega. Pri preverjanju, ali je mogoča katera koli od teh potez, potrebujemo podatke o trenutnem igralcu, prostih poljih med trdnjavo in kraljem, pravicah do izvajanja rokade in napadu na določena polja med trdnjavo in kraljem.

- **En passant:** pri generiranju takšnih potez potrebujemo poleg stanja pozicije še informacijo o tem, na katerem polju je en passant mogoč. Polje en passant se spreminja med potekom igre in se nastavi v primeru dvojnega premika kmeta. Polje shrani pozicijo tako, kot da bi se kmet premaknil samo za eno polje, kar pomaga pri detekciji s pomočjo napadalnih tabel. V primeru, da je en passant mogoč, nastavimo še primerno zastavico za lažje zaznavanje pri igranju in izničenju potez.

4.6 Testiranje pravilnosti generiranih potez

Po implementaciji generiranja vseh psevdo legalnih potez je zelo pomembno testiranje pravilnosti. Pravilnost generiranih potez je pomembna, ker lahko v primeru generiranja nepravilne poteze pride do uničenega stanja pozicije, ki pozneje vpliva na ocenjevanje pozicije ali privede celo do priporočila nelegalnih potez. Da bi se temu čim bolj izognili, smo izdelali teste za vse figure in posebne možne poteze. Za vsako figuro smo pripravili teste za preverjanje pravilnega števila možnih potez v primeru neblokiranega možnega premikanja, blokirano premikanje zaradi robov šahovnice, blokirano premikanje zaradi drugih figur in večje število enakih figur na šahovnici. Posebej so bili dodani še testi za promocije kmeta, en passant in rokade.

4.7 Igranje potez

Po izdelavi testov in testiranju pravilnosti je naslednji korak implementacija igranja potez. Poleg generiranja potez je to ena izmed funkcionalnosti, ki se največ uporablja med analizo pozicije, zato moramo zagotoviti, da je

igranje potez kar se da hitro. Zaradi izbire shranjevanja figur in pozicije v bitnih tabelah se v večini primerov implementacija igranja potez razbije na dve operaciji. Ena operacija izvede odstranjevanje figure z določenega polja, in sicer s pomočjo operacije NOT AND nad potrebnimi bitnimi tabelami. Druga operacija izvede dodajanje figure na določeno polje, in sicer s pomočjo operacije OR. Pri tem moramo poskrbeti za skladnost med bitnimi tabelami. V primeru belega skakača moramo odstranjevanje in dodajanje polja izvesti nad bitno tabelo belih skakačev, belih in vseh figur, saj lahko sicer pride do nepravilnega stanja šahovnice, kar lahko pozneje povzroči generiranje in igranje potez, ki ne obstajajo. To na koncu pripelje do priporočanja ilegalnih potez. Poleg premikanja figur moramo poskrbeti še za nekatere posebnosti med igro.

4.8 Odigraj potezo

V naši implementaciji imajo figure med seboj zelo podobno strukturo igranja potez, ampak obstaja tudi dovolj posebnih primerov. Deli igranja potez:

- **Premikanje figur:** kot že omenjeno, postopek premikanja figur na šahovnici temelji na odstranjevanju in dodajanju figure na določeno polje z operacijama NOT AND in OR. To je osnova, na kateri se pozneje gradijo še dodatne izjeme.
- **Jemanje figur:** pri jemanju figur moramo biti pozorni, da poleg spreminjanja bitnih tabel figure, igralca in vseh figur, spremenimo še pravilno tabelo nasprotnikove vzete figure. Informacija o vzeti figuri je shranjena v zastavice poteze že pri generiranju potez.
- **Promocija kmeta:** pri promociji moramo imeti informacijo, v katero figuro kmeta promoviramo. To je v našem primeru zastavica, ki se nastavi pri generiranju poteze. Po pridobljeni informaciji moramo pravilno odstraniti kmeta iz bitne table in dodati promovirano figuro v pravilno bitno tabelo na enako mesto, kamor se je premaknil kmet.

- **En passant:** v primeru premika kmeta za dve polji moramo poskrbeti za pravilno postavitev en passant polja, ki se uporablja v nadaljnjih potezah za detekcijo možnega jemanja kmeta na način en passant. Po naslednji odigrani potezi moramo polje en passant na novo postaviti, saj je jemanje en passant mogoče samo v naslednji potezi. V primeru jemanja moramo poskrbeti za detekcijo, da se je en passant resnično zgodil in temu pravilno odstraniti kmeta, saj ne leži na enakem polju kot kmet, ki ga je vzel. Detekcija poteze en passant je narejena s pomočjo zastavice pri generiranju potez.

- **Rokada:** tukaj moramo biti pozorni na pravilno odvzemanje pravic izvajanja kratke in dolge rokade. Pravico do rokade odvzamemo samo v primeru, da pravico v trenutni poziciji imamo in odigramo potezo s kraljem ali trdnjavo. Pravico moramo odvzeti tudi v primeru, ko nam nasprotnik vzame trdnjavo. V primeru premiku kralja odvzamemo pravico do kratke in dolge rokade, če je to mogoče. V primeru premika ali odvzema trdnjave pa odvzamemo samo pravico do rokade za tisti del šahovnice, na katerem je bila trdnjava. Odvzem pravice se nastavi kot zastavica v potezi in naknadno spremeni še pravice na šahovnici. Nato ne smemo pozabiti na spremembo pozicije trdnjave ob igranju rokade.

4.9 Izniči potezo

Po implementirani funkcionalnosti igranja potez potrebujemo še funkcionalnost izničenja potez. Ta operacija služi kot popolnoma obratna operacija igranja potez, kar pomeni, da mora stanje šahovnice ostati enako po izvedbi para funkcij igraj in izniči potezo. V primeru, da se stanje šahovnice po izvedbi para funkcij spremeni, pomeni nepravilno delovanje ene izmed funkcij, kar lahko pripelje do uničenega stanja šahovnice. Deli izničenja poteze:

- **Premikanje figur:** potek premika figure je enak kot pri igranju poteze, ampak z obrnjenimi operacijami odvzema in dodajanja figure. Če smo

pri igranju poteze odstranili figuro iz določenega polja in jo dodali na novo polje, jo sedaj odstranimo iz tistega polja in jo dodamo nazaj na originalno polje.

- **Jemanje figur:** pri izničenju jemanja figur moramo dodati nazaj pravilno figuro, ki smo jo prvotno vzeli. Kot že omenjeno, imamo to informacijo shranjeno v zastavici poteze.

- **Promocija kmeta:** paziti moramo na pravilen odvzem promovirane figure in dodajanje kmeta na prvotno mesto pred promocijo. Informacijo o promovirani figuri hranimo v zastavici poteze.

- **En passant:** pri izničenju polja en passant moramo paziti na pravilno ponastavitev polja, saj izničenje poteze ne pomeni, da je bilo prejšnje polje en passant prazno. V primeru premika več kmetov za dve polji zaporedoma, se polje en passant pravilno ponastavi po vsaki izničitvi poteze. V primeru jemanja en passant pa moramo pravilno dodati nazaj vzetega kmeta.

- **Rokada:** tako kot pri izničenju en passant, moramo pravilno ponastaviti pravice kratke in dolge rokade. Pomagajo nam zastavice o pravicah, ki smo jih nastavili med igranjem poteze.

4.10 Testiranje pravilnosti igranja potez

Po implementaciji igranja in izničenja potez sledi dodajanje testov pravilnosti. Takšni testi so ključni pri izdelavi programa za analizo šahovskih pozicij. Brez testov oziroma s slabimi testi hitro pride do napak, kot je uničeno stanje šahovnice. V primeru uničenega stanja šahovnice začnemo izvajati generiranje ilegalnih potez, ocenjevanje ilegalnih pozicij, priporočanje ilegalnih potez. Takšne napake je pozneje težko oziroma skoraj nemogoče odkriti in popraviti. Zato smo implementirali teste igranja in izničenja potez, ki preverjajo pravilno število odigranih potez na določeni globini na začetni poziciji. Po vsakem

paru operacij igranja in izničenja poteze se preveri enako stanje šahovnice pred in po izvedbi operacij. Ker pa to ne vključuje primerov, kot so testiranje promocije in rokade, saj zaradi velikega faktorja razvejanosti izvajamo teste do globine 6, izvajamo še teste na pozicijah, ki so namenjeni testiranju rokade in promocije.

Poglavje 5

Iskanje najboljše poteze

Po implementaciji generatorja potez, funkcionalnosti igranja in izničevanja potez imamo pripravljene vse elemente programa za začetek preiskovanja vseh možnih linij igre. Naslednji korak je implementacija igranja vseh linij in določanje najboljše poteze. Iskanje najboljše poteze je izvedeno s pomočjo algoritmov, kot so minimax z alfa beta rezanjem [2], negamax [1], iterativno poglabljanje, iskanje quiescence [2] in mnogih iskalnih operacij. Večino omenjenih algoritmov se uporablja skupaj, ampak vsak izmed njih rešuje svoj specifičen izziv. Preiskovanje potez je zelo zahteven izziv, saj ima eksponentno časovno zahtevnost. Za prikaz njegove zahtevnosti vzemimo 20 možnih potez, kar je število možnih potez začetne pozicije, ki jih moramo pregledati na vsaki globini.

$$f(d) = 20^d$$
$$f(1) = 20^1 = 20 \text{ potez}$$
$$f(2) = 20^2 = 400 \text{ potez}$$
$$f(3) = 20^3 = 8\,000 \text{ potez}$$
$$f(4) = 20^4 = 160\,000 \text{ potez}$$
$$f(5) = 20^5 = 3\,200\,000 \text{ potez}$$
$$f(6) = 20^6 = 64\,000\,000 \text{ potez}$$
$$f(7) = 20^7 = 1\,280\,000\,000 \text{ potez}$$

Tukaj lahko vidimo, da število potez enormno narašča. Prikazani izračuni uporabljajo dokaj majhen faktor razvejanosti, saj se ta zelo poveča ob odprtju pozicije. Zaradi težavnosti velikega števila potez moramo med iskanjem poskrbeti za čim manjši faktor razvejanosti, saj je to edini resen način doseganja večjih globin v poziciji, ki pripomorejo k bolj natančni oceni pozicije.

5.1 Iskanje po vseh možnih potezah

Pri glavnem delu iskanja obstajata dva glavna algoritma, ki opravljata to nalogo, in sicer:

- **Algoritem minimax z alfa beta rezanjem [2]:** algoritem deluje na principu maksimiziranja in minimiziranja ocen pozicije. Pri šahu se po navadi uporablja maksimiziranje za belega igralca in minimiziranje za črnega igralca. V osnovi algoritem deluje na način iskanja v globino. Alfa beta rezanje pa se uporablja kot iskalna optimizacija algoritma, saj zmanjša faktor razvejanosti, kar zelo pripomore k doseganju višjih globin. Alfa beta rezanje deluje tako, da med iskanjem shranjuje vrednosti v spremenljivki alfa in beta. Če pri maksimiziranju dobimo vrednost, večjo kot je bila prejšnja najboljša poteza, jo shranimo v spremenljivko alfa. V primeru minimiziranja pa najboljšo potezo shranimo v spremenljivko

beta. Če med iskanjem alfa naraste na večjo vrednost kot beta, lahko za tisto vozlišče prenehamo z iskanjem novih potez, saj vemo, da nadaljnje iskanje ne bo spremenilo izida najboljše izbrane poteze v enem izmed prejšnjih vozlišč. Prednost tega algoritma je delitev funkcionalnosti na preiskovanje potez belega in črnega igralca.

- **Algoritem negamax z alfa beta rezanjem [1]:** algoritem deluje na principu maksimiziranja za oba igralca. Tako kot minimax, ta algoritem deluje na način iskanja v globino. Tudi način rezanja vozlišč ostane enak. Od algoritma minimax se razlikuje po načinu shranjevanja ocene pozicije in spreminjanja alfa in beta spremenljivke. V osnovi je ocena šahovske pozicije pozitivna, če je ocenjena bolje za belega igralca in negativna, če je ocena boljša za črnega igralca. Ker ta algoritem uporablja princip maksimiziranja za obe strani, moramo ocenjeno pozicijo v primeru belega igralca pustiti nespremenjeno, v primeru črnega igralca pa to vrednost negirati. Vzemimo primer ocene pozicije 5. Za belega igralca pomeni to zelo dobra pozicija, za črnega pa zelo slaba. Zato v primeru ocenjevanja s strani črnega igralca to oceno pretvorimo v –5. Spremeni se tudi potek shranjevanja alfa in beta spremenljivke. Med izvajanjem algoritma se zaradi konstantnega maksimiziranja spreminja samo spremenljivka alfa. Da dobimo nazaj željeno delovanje rezanja pozicije, moramo med vsakim klicem naslednjega preiskovanja obrniti spremenljivki alfa in beta, kar ponazori sprememba barve igralca. Prednost tega algoritma je izdelava samo ene funkcije, ki skrbi za oba igralca in jo je dolgoročno lažje spreminjati in vzdrževati.

V naši implementacijo smo se najprej odločili za uporabo algoritma minimax zaradi manjše kompleksnosti in razbitja na preiskovanje belega in črnega igralca. Pozneje pa smo to prepisali na algoritem negamax zaradi lažjega vzdrževanja in dodajanja novih optimizacij.

5.2 Preverjanje legalnosti poteze

Po implementaciji iskanja negamax nam manjka samo še ena funkcionalnost za pravilno delovanje simulacije igre. To je preverjanje legalnosti potez. Kot že omenjeno, naša implementacija uporablja generiranje psevdo legalnih potez, kar pomeni, da vse generirane poteze niso legalne, zato moramo preverjanje legalnosti izvesti med preiskovanjem potez znotraj iskanja negamax. Če preverjanje legalnosti izpustimo, lahko pride do potez, kot je jemanje kralja, ki pa v naši implementaciji povzroči nedefinirano stanje pozicije, saj program v vseh primerih pričakuje, da imata obe strani prisotnega kralja. Zaradi izbire shranjevanja pozicij in figur v bitne tabele, postane preverjanje legalnosti hitra in lahka operacija. Najprej odigramo generirano potezo in nato preverimo legalnost pozicije. Če naš kralj ni v šahu, pomeni da je pozicija legalna, če šah je prisoten, to pomeni, da pozicija ni legalna. Za preverjanje legalnosti imamo dve možnosti. Prva možnost je, da preverimo vse figure, pridobimo maske možnih napadov, ki jih imamo shranjene v napadalnih tabelah in preverimo, ali se katero od napadenih polj ujema s poljem kralja. Ta način preverjanja je potraten, saj moramo za vsako figuro preveriti prisotnost šaha. Tako dobimo dodatno informacijo, katera figura napada kralja, ampak mi te informacije ne potrebujemo. Drug način je način, ki smo ga uporabili v naši implementaciji. To je preverjanje, ali lahko kralj napade nasprotnikove figure, če se obnaša enako kot tisti tip figure. Pri šahu za vse figure velja pravilo simetrije, razen pri jemanju en passant, ki nima vpliva na zaznavanje šaha. Če naša figura napada enak tip nasprotnikove figure, pomeni, da nasprotnikova figura napada tudi našo figuro. Zato lahko generiramo maske možnih napadov za vse tipe figur za polje, na katerem stoji kralj. Ko imamo maske generirane, primerjamo prisotnost nasprotnikovih figur različnih tipov s polji v maski. Če se katerokoli izmed polj ujema, pomeni, da je naš kralj v šahu in pravkar odigrana poteza ni legalna.

5.3 Iterativno poglabljanje

Po izdelanem glavnem iskalnem algoritmu negamax imamo narejeno osnovo za preiskovanje najboljših potez. Algoritem ima eno slabost, ki je vnaprej določena globina. Možna dosegljiva globina v določenem času se zelo razlikuje glede na pozicijo, ki jo analiziramo. Analiziranje zelo odprte pozicije z mnogo figurami traja na določeni globini mnogo dlje kot pozicija z manj figurami, kar pomeni, da lahko preiskovanje globine poteka dlje, kot imamo na voljo časa, ali pa v primeru majhnega števila figur ostane veliko neporabljenega časa, ki bi ga lahko porabili za preiskovanje večjih globin. V ta namen se uporablja algoritem iterativnega poglabljanja. Iterativno poglabljanje deluje tako, da začne preiskovati do globine 1. Ko globino preišče, shrani najboljšo potezo te globine in začne od začetka preiskovati vse poteze še enkrat, ampak do globine 2. To se nadaljuje, dokler nam ne zmanjka časa, ki je vnaprej določen za vsako potezo. Takšen način preiskovanja pridobi tudi nekaj zgodovinskih informacij, ki so uporabne za nadaljnje optimizacije preiskovanja. Čeprav se ta način preiskovanja zdi zelo potraten, saj vsakič preverjamo enake poteze, kot smo jih doslej, v resnici ne opravimo veliko več dela za zgoraj omenjene koristne lastnosti algoritma. Za primer dodatnega dela lahko vzamemo zgornji primer števila potrebnih pregledanih potez, kjer lahko vidimo, da je potrebno opravljeno delo za pregled vseh potez na globini 6 bistveno manjše od potrebnega dela za pregled vseh potez na globini 7.

5.4 Iskanje quiescence

Na tej točki imamo način za preiskovanje in shranjevanje najboljše poteze na zadnji preiskani globini. Čeprav nismo več omejeni na specifično globino celotnega preiskovanja, smo omejeni na specifično globino trenutnega preiskovanja. Tukaj nastane težava, saj se lahko preiskovanje pozicije konča na sredini taktičnega zapleta. Pod taktični zaplet štejemo možno jemanje figur, promocijo kmeta in obstoječi šah. V primeru jemanja in promocije figur se zaradi takšnega iskanja ocenjevanje pozicije zelo nagne v prid igralca, ki je

odigral zadnjo takšno potezo. To pripelje do spregleda taktik, ignoriranja najboljših možnih potez in priporočanja slabih potez. V izogib takšnim težavam uporabimo algoritem iskanja quiescence [2]. Ta algoritem v osnovi deluje zelo podobno kot negamax, ampak z nekaterimi pomembnimi razlikami. Oba algoritma delujeta na principu maksimiziranja pozicije. Uporabljata enako logiko shranjevanja in rezanja s pomočjo spremenljivk alfa in beta. Glavne razlike algoritmov:

- **Preiskovanje potez:** negamax preiskuje vse možne poteze v pozicije, kar povzroči visok faktor razvejanosti, ampak s tem ocenjuje bolj natančno. Iskanje quiescence preiskuje vse poteze samo v primeru, ko je v poziciji prisoten šah. V primerih, kjer šaha ni, preiskuje samo tiste poteze, ki bi povzročile jemanje in promocijo figur.

- **Globina iskanja:** negamax ima vnaprej določeno globino, do katere lahko preiskuje poteze. Iskanje quiescence pa ima neomejeno globino preiskovanja. Preiskovanje se konča v primeru, ko taktičnih potez zmanjka, pozicija nima več legalnih potez, kar v primeru, da je v trenutni poziciji prisoten šah, pomeni mat in v primeru, ko ocenimo, da je pozicija dovolj dobra, tudi če ne odigramo naslednje poteze.

- **Prisotnost remija in mata:** pri algoritmu negamax lahko brez težav ugotovimo prisotnost mata in remija v določeni poziciji, saj preiskujemo vse možne poteze. V primeru, ko v trenutni poziciji legalne poteze ne obstajajo, vemo, da je prišlo do pata ali mata. V tem primeru moramo preveriti, ali je kralj v šahu ali ne. Če je, to pomeni, da je na šahovnici prisoten mat. Če legalnih potez ni in šah ni prisoten, to pomeni, da smo prišli do pata. Do remija lahko pridemo tudi v primeru trojnega ponavljanja, ki pa se išče na drugačen način. Kot pri iskanju negamax, lahko pri iskanju quiescence opazimo mat na enak način. Problematično postane iskanje pata, saj quiescence ne preiskuje vseh potez. V primeru, da v poziciji ni taktičnih legalnih potez, to še ne pomeni, da druge legalne poteze ne obstajajo. Zato se takšna detekcija pata v iskanju

quiescence ne izvaja. V naši implementaciji uporabljamo iskanje remija samo v primerih trikratne ponovitve. To preiskovanje remijev bi lahko tudi izpustili, saj se pri iskanju quiescence redko zazna.

- **Ocenjevanje pozicije:** negamax izvede oceno pozicije, ko pride do vnaprej določene globine. Iskanje quiescence pa izvede ocenjevanje pozicije na vsakem vozlišču in preneha s preiskovanje potez v primeru, da je pozicija dovolj dobra.

Zaradi mnogih prednosti in slabosti obeh algoritmov, se ne izbere le enega izmed njih, ampak se uporabljata skupaj. Ko pri iskanju negamax naletimo na globino, pri kateri bi se morali ustaviti in vrniti najboljšo najdeno potezo, se namesto ocene pozicije izvede iskanje quiescence, ki preišče vse možne taktične poteze, ki ji negamax ni mogel pregledati.

5.5 Preverjanje trikratne ponovitve pozicije

Zadnji korak, pred zaključkom osnovne implementacije iskanja potez, je preverjanje remija v primeru trikratne ponovitve pozicije. Preverjanje remija v tem primeru je pomembo, da se program ne vrti v krogu kombinacije potez, ki bi povzročile remi v dobri poziciji. Čeprav program oceni pozicijo z visoko oceno v primeru trikratne ponovitve, moramo oceno spremeniti na vrednost remija, ki je v naši implementaciji 0. Za preverjanje remija uporabljamo sklad, ki hrani zgoščene vrednosti pozicij, ki so se zgodile med igro, mesto zadnje poteze, ki povzroči, da je trikratna ponovitev nemogoča in število odigranih potez. Poteze, ki povzročijo, da trikratna ponovitev ni več mogoča, so poteze jemanja in premiki kmetov. Po vzetju figure v nadaljevanju ne moremo več preiti v pozicijo, ki bi se ponovila, saj figura ne obstaja več. Kmeti so edine figure, ki se po premiku naprej ne morejo več vrniti nazaj, zato tudi premiki kmetov povzročijo pozicije, na katerih je trikratno ponavljanje za nazaj nemogoče. Preverjanje ponavljanja pozicije poteka tako, da preverimo, ali se je trenutna pozicija, ki je predstavljena v zgoščeni vrednosti, ponovila

trikrat od najnovejše igrane poteze do poteze, ki je povzročila nemogoče ponavljanje za nazaj.

Poglavje 6

Ocena pozicije

Zadnja funkcionalnost, ki nam na tej točki razvoja manjka, je funkcionalnost ocenjevanja posamezne pozicije, da dobimo osnovno verzijo delujočega programa za analiziranje šahovskih pozicij. Za ocenjevanje pozicij uporabljamo različne hevristike. Pri hevristikah poskušamo dobiti čim bolj realne ocene pozicij, ampak še vseeno želimo, da se izračun ocene zgodi čim hitreje, saj se ocenjevanje dogaja v vsakem vozlišču pri iskanju quiescence. V naši implementaciji smo za hevristike uporabili nekaj osnovnih pravil iz teorije šaha. Ta pravila so: ocena materiala, mobilnosti in pozicija figur ter varnost kralja.

6.1 Ocena materiala

Ocena materiala je tista ocena, ki v večini primerov najbolj pripomore k oceni pozicije. Ta ocena se izračuna iz števila figur, ki jih imata oba igralca v poziciji, ki se ocenjuje. Za vsak tip figure imamo vnaprej definirano vrednost.

$$\text{Material} = \sum_{f \in Figure} \check{S}tevilo_f * Vredost_f$$

$$\text{OcenaMateriala} = Material_{beli} - Material_{črni}$$

6.2 Ocena mobilnosti

Pri oceni mobilnosti uporabljamo približno oceno s pomočjo napadalnih mask, ki so shranjene v napadalnih tabelah. Naša implementacija za oceno mobilnosti ne uporablja kmetov in kralja, saj nam mobilnost teh figur težko doda pravilno informacijo o kakovosti pozicije. Pri izračunu uporabljamo koncept psevdo legalnih premikov, saj bi bilo generiranje in preverjanje vseh možnih potez v poziciji zelo potratno. Tako kot pri oceni materiala, uporabljamo različne vnaprej definiranje vrednosti za različne tipe figur glede na število možnih potez.

6.3 Ocena pozicije figur

Pri oceni pozicije figur uporabljamo vnaprej definirane pozicijske tabele [3]. Pozicijska tabela je tabela, ki hrani bonuse in kazni, glede na to, na katerem polju stoji katera figura. Za primer lahko vzamemo skakača, ki je v veliki večini pozicij bolj uporaben v središču šahovnice kot na robu. Poleg pozicijskih tabel upoštevamo še odprte in pol odprte linije trdnjav, ki dodajo bonus oceni pozicije. Pri pozicijah kmetov upoštevamo podvojene kmete in proste kmete. V primeru podvojenih kmetov pozicijo rahlo kaznujemo. V primeru prostih kmetov pozicijo rahlo nagradimo.

6.4 Ocena varnosti

Pri oceni varnosti pozicije upoštevamo varnost kralja. V naši implementaciji nagradimo igralca, ki je izvedel rokado. Poleg same rokade kaznujemo in nagrajujemo še pozicijo kmetov pred kraljem v primeru rokade. V primeru, da je kralj odprt oziroma nima povezane obrambe kmetov pred seboj, se pozicijo kaznuje, v nasprotnem primeru se jo nagradi. Imamo dve vrsti nagrad oziroma kazni, glede na to, v kateri vrsti so kmeti postavljeni pred kraljem. Če so kmeti prisotni eno vrsto pred kraljem, dobi pozicija največjo možno nagrado. Če pa so kmeti prisotni dve vrstici pred kraljem, še vseeno dobijo

nagrado, ampak manjšo. Takšne ocene se ne izvajajo v končnicah, saj je v večini končnic aktiven kralj bolj pomemben od varnosti, saj je odprtega kralja sredi šahovnice težje izkoristiti kot v sredini igre. Za zaznavanje končnice v naši implementaciji uporabljamo oceno nasprotnikovega materiala. Od ocene odštejemo vrednost kmetov in preverimo vrednost preostale ocene. V primeru, da ocena presega približno vrednost dveh trdnjav in enega lovca, štejemo pozicijo kot končnico.

Poglavje 7

Iskalne optimizacije

Po implementaciji ocene pozicij imamo popolnoma delujoč program za analizo šahovskih pozicij, kateremu podamo pozicijo in dobimo nazaj najboljšo možno potezo ter oceno te poteze po zgoraj omenjenih hevristikah. Čeprav imamo popolnoma delujoč program, še vedno obstaja težava pregledovanja pozicij do premajhne globine. Manjša kot je preiskana globina, manj natančno program oceni pozicijo, saj lahko izračuna samo majhno število potez vnaprej. Kot smo že pokazali, je težava preiskovanja večjih globin velik izziv, saj je eksponenten. Edini resen način za doseganje večjih globin je preiskovanje manj potez na vsakem vozlišču. Tukaj nastopijo tehnike iskalnih optimizacij.

7.1 Transpozicijske tabele

Ena izmed največjih optimizacij, ki smo jo dodali programu, je transpozicijska tabela [9]. Transpozicija v šahu se zgodi takrat, ko pridemo do enake pozicije po različnih odigranih potezah. Pri programih za analiziranje pozicij pride do transpozicij kar pogosto, saj pregledujemo vse možne poteze, kar pomeni, da že spremenjen vrsti red enakih potez pripelje do transpozicij. Ideja transpozicijske tabele je vračanje najboljše poteze ter njene ocene, če smo to pozicijo že srečali pri pregledu drugih potez. Prvi izziv, s katerim se srečamo, je način shranjevanja pozicij, ki smo jih že pregledali. V šahu lahko rečemo,

da ja pozicija popolnoma enaka drugi poziciji, če izpolnjuje štiri pogoje:

1. **Igralec na potezi:** v obeh pozicijah mora biti enak igralec na potezi.
2. **Figure:** v obeh pozicijah morajo biti figure obeh igralcev na enakih poljih.
3. **Pravice do izvajanja rokade:** v prvi poziciji morata oba igralca imeti enake pravice do izvajanja rokade kot v drugi poziciji.
4. **Polje en passant:** v obeh pozicijah se morata ujemati polji, na katerih je možno jemanje en passant.

Preverjanje vseh navedenih pogojev vsakič, ko želimo preveriti, ali je pozicija transpozicija, bi bilo računsko zelo potratno. V ta namen se uporabljajo zgoščene vrednosti pozicije. Te vrednosti so izračunane z zgoščevanjem Zobrist [16]. Zgoščevanje Zobrist uporablja tehniko zgoščevanja XOR. Zgoščevanje bitnih tabel z operacijo XOR ni primerno, saj bi zamenjava dveh figur na šahovnici pomenila kolizijo, kar pa pripelje do enake ocene dveh popolnoma drugačnih pozicij. Zato zgoščevanje Zobrist uporablja naključno generirana števila. Za vsak tip figure na vsakem polju se določi naključno generirano 64-bitno število, ki se uporablja za namen zgoščevanja XOR. Enako se generirajo tudi števila za barvo igralca, ki je trenutno na potezi, pravice do rokade in možno polje en passant. Zgoščena vrednost se gradi med izvajanjem igranja in izničenja poteze. Zgoščena vrednost pozicije se hrani v šahovnici. Vzemimo primer premika skakača. Ko odstranimo skakača z določenega polja, izvedemo operacijo XOR z naključnim številom, ki smo ga generirali za skakača na tistem polju. Enako storimo tudi za premik skakača na določeno polje. Upoštevati moramo tudi vse izjeme, ki spremenijo pozicijo, kot so jemanje figur, promocija, rokada in jemanje en passant ter ustrezno spremeniti zgoščeno vrednost pozicije. Zgoščene vrednosti se uporabljajo tudi pri prepoznavanju trikratne ponovitve pozicije. Ko imamo računanje zgoščenih vrednosti implementirano, je treba poskrbeti za strukturo, ki se bo uporabljala za shranjevanje in detekcijo transpozicij. V naši implementaciji

shranjujemo zgoščeno vrednost, globino, na kateri je bila najboljša poteza ocenjena, zastavico, starost in potezo. Za shranjevanje transpozicij se uporablja tabela 2^n velikosti, kjer n predstavlja število bitov, ki se uporabljajo iz zgoščene vrednosti pozicije za indeksiranje tabele. Shranjevanje transpozicij vsebuje določene pogoje. Po pregledanem vozlišču se najboljša poteza v poziciji poskuša shraniti v transpozicijsko tabelo. Poteza ima lahko eno izmed treh tipov ocen. Če vrednost ocene pade med prvotni vrednosti alfe in bete, ki smo ju prejeli ob vhodu v vozlišče, to pomeni, da je ocena za pozicijo točna. Če ocena pade pod vrednost prvotne alfe, to pomeni, da je ocena pozicije največ tako dobra. Če ocena pade nad vrednost prvotne bete, to pomeni, da je ocena vsaj tako dobra. Navedene tri primere dobimo zaradi alfa beta rezanja, ki se izvaja v algoritmu negamax. V nekaterih pozicijah prenehamo s poskušanjem novih potez, saj glede na prejšnje podatke o vrednostih alfe in bete nadaljnje preiskovanje ne bi spremenilo rezultata, zato za nekatere preiskane pozicije ne vemo točne ocene, saj je možno, da najboljše poteze v tisti poziciji še nismo preiskali. Navedeni trije primeri se shranijo v zastavico med shranjevanjem transpozicije. Če je mesto, na katerega shranjujemo transpozicijo, prazno, jo shranimo. V primeru, da pride do kolizije v tabeli, se v naši implementaciji preverita dva pogoja: ali je zapis star, kar pomeni, da je bil vnesen pred izvedbo zadnje poteze; ali je nova pozicija pregledala več potez vnaprej. Če katerikoli pogoj drži, se zapis prepiše. Preverjanje staranja transpozicijskih tabel je pomembno, ker brez njega v transpozicijski tabeli ostanejo zapisi, ki jih ne moremo nadomestiti, ker so pregledali visoko globino in do teh pozicij v veliko primerih ne moremo več priti zaradi izvajanja potez, ki preprečijo prehod v prejšnje stanje, kot so premiki kmetov, jemanje figur in promocija kmetov. Manj prostora kot imamo v transpozicijski tabeli, manjša je verjetnost, da bomo transpozicijo zadeli. Preverjanje zadetka transpozicije postane trivialno. Če v transpozicijski tabeli obstaja zapis z enako zgoščeno vrednostjo in pogoji za vračanje držijo, ga vrnemo. Oceno pozicije lahko vrnemo samo v primeru, ko je pregledana globina zapisa večja ali enaka trenutni preostali globini pozicije, ki jo ocenjujemo. Pri vračanju najboljše

poteze moramo biti pozorni tudi na zastavico, ki je nastavljena v transpoziciji. V primeru, da je zastavica nastavljena na točno vrednost pozicije, jo lahko vrnemo. Če gre za zastavico, ki nam pove, da je poteza največ toliko dobra, kot je zapisano v transpoziciji, mora veljati še pogoj, da je ocena pozicije manj vredna od trenutne vrednosti alfe, da lahko oceno vrnemo. Podobno je pri zastavici, ki nam pove, da je pozicija najmanj tako dobra, kot je zapisano v transpoziciji. V tem primeru mora biti ta ocena večja od trenutne vrednosti bete, da lahko vrednost vrnemo. Uporaba transpozicijske tabele pri iskanju quiescence ima eno posebnost. V naši implementaciji se transpozicije pri iskanju quiescence shranijo z globino –1, kar pomeni, da jih lahko zamenja katera koli nova pozicija v primeru kolizije. To se uporablja, ker so zapisi iz iskanja negamax več vredni kot iskanja quescence.

7.2 Urejanje potez

Urejanje potez [1] je ena izmed optimizacij, ki je zelo povezana z drugimi iskalnimi optimizacijami. Ideja je, da namesto preizkušanja psevdo naključnih potez uredimo poteze po nekem kriteriju, saj preizkušanje boljših potez najprej pripelje do večjega števila rezanj vozlišč. Seveda pa brez ocenjevanja potez ne moremo vedeti, katera poteza je najboljša. Zato se uporabljajo različne hevristike za oceno vrstnega reda potez v naslednjem vrstnem redu:

1. **Transpozicijska tabela:** če je možno, za prvo najboljšo potezo izberemo tisto, ki je shranjena za trenutno pozicijo v transpozicijski tabeli. Pri tem se pokaže koristna lastnost iterativnega poglabljanja, saj nam to informacijo pridobi iz prejšnjega iskanja, ki je za eno globino manj globoko. Velika verjetnost je, da bo prejšnja pojavitev najboljše poteze te pozicije, ena izmed najboljših potez tudi pri preiskovanju enake pozicije pri povišani globini.

2. **Promocije:** če v transpozicijski tabeli ni primernega zapisa, se pregleda možnost promocije. Če je promocija v poziciji mogoča, se preizkusi najprej.

3. **Jemanje figur:** če promocija ni mogoča, se pregledajo vse poteze jemanja figur in se uredijo po načinu MVV-LVA. Pri tem načinu najprej preverimo poteze, ki vzamejo nasprotnikovo najbolj vredno figuro z našo najmanj vredno figuro.

4. **Ubijalske poteze [2]:** ubijalske poteze so poteze, ki v algoritmu negamax povzročijo alfa beta rezanje in ne vsebujejo jemanja ali promocije figur. V naši implementaciji se shranijo dve taki potezi za vsako globino. V primeru kolizije se starejša poteza zamenja z novo. Ubijalske poteze delujejo po principu lokalnosti. Če je na globini odkrita takšna poteza, obstaja realna verjetnost, da bo v drugi poziciji na enaki globini ta poteza dobra.

5. **Preostale poteze:** preostale poteze se razvrstijo s pomočjo zgodovinske hevristike [2]. Podobno kot pri ubijalskih potezah, zgodovinska tabela hrani podatke o potezah, ki so povzročile alfa beta rezanje pri potezah, ki ne vsebujejo jemanja in promocij. Shranjevanje se bistveno razlikuje od shranjevanja ubijalskih potez. Zgodovinska tabela v naši implementaciji za shranjevanje ocene potez potrebuje tri podatke - polje, iz katerega se figura premakne, polje, na katero se figura premakne in katera figura se je premaknila. V primeru alfa beta rezanja dodamo oceni na mestu v tridimenzionalni tabeli $PreostalaGlobina^2$ ocene. Vsem prejšnjim potezam, ki ne vsebujejo jemanja in promocije, pa odvzamemo vrednost $PreostalaGlobina$. Pri sortiranju ostalih potez uporabimo vrednosti iz zgodovinske tabele za določitev vrstnega reda.

Naša implementacija za sortiranje uporablja način lenega sortiranja, kar pomeni, da pridobimo naslednjo najboljšo potezo šele, ko prejšnjo preverimo, saj bi bilo urejanje celotnega seznama dokaj potratno, še posebej, ker veliko potez zaradi rezanja ne bo nikoli prišlo na vrsto.

7.3 Rezanje null move

Naslednja iskalna optimizacija, ki smo jo implementirali, je razanje null move [4]. Rezanje null move se izvaja v algoritmu negamax. Ideja te optimizacije je, da pridobimo informacijo, ali je trenutna pozicija še vedno dovolj dobra, če bi našo potezo preskočili. V primeru, da je pozicija dovolj dobra, naknadno iskanje potez ni potrebno, saj s tem pridobimo informacijo, da v veliki večini primerov obstaja vsaj ena poteza v poziciji, ki bi obdržala ali celo izboljšala oceno pozicije. Za uporabo rezanja null move obstajajo določeni pogoji, ki jih moramo upoštevati. Prvi pogoj je, da v vozlišču in vseh njegovih podvozliščih, pred katerimi smo uporabili null move, ne smemo ponovno uporabiti poteze null move, saj bi to pripeljalo samo do podajanja poteze. Drugi pogoj je, da se null move ne izvede v primeru ocenjevanja potez na globini 1 in v primeru, da je globina do konca preiskovanja premajhna - v naši implementaciji, če je preostala globina manjša od 3. To storimo, ker pri globini 1, kjer je alfa najmanjša možna vrednost in beta največja možna vrednost, ne moremo dobiti ocene pozicije, ki je večja od vrednosti bete. Zato je takšno preiskovanje nesmiselno. Drugi pogoj pa uporabimo, ker ne želimo prehitro preskočiti v preiskovanje taktičnih potez. Naslednji pogoj je, da trenutno naš kralj ne sme biti v šahu, saj bi igranje nasprotnikove poteze povzročilo jemanje kralja, kar je nedefinirano obnašanje v našem programu. Dodali smo še pogoj, da se null move lahko izvaja samo v primeru, če so v poziciji prisotne druge figure kot kralj in kmeti, saj lahko v takšnih končnicah pride do napačne ocene pozicije. Poleg pogojev se rezanje null move razlikuje še v načinu iskanja. Namesto navadnega iskanja, ki bi bilo pri višjih globinah zelo potratno, se tukaj uporablja zmanjšana preiskava globine. V naši implementaciji zmanjšamo iskanje za tri poteze. Poleg zmanjšane globine se uporablja tudi spremenjeno okno alfe in bete. Ker nas zanima samo, ali je poteza dovolj dobra in ne točna vrednost, se tukaj uporablja okno [–beta, –beta + 1]. Sprememba predznakov se uporablja zaradi algoritma negamax. V primeru, da to iskanje vrne vrednost, ki je dovolj dobra, namesto iskanja po vseh poteza, vrnemo originalno vrednost bete, ki pomeni, da je pozicija

vsaj toliko dobra, kot je vrednost bete. Če ne dobimo dovolj dobre pozicije, nadaljujemo s pregledovanjem vseh možnih potez.

7.4 Iskanje PVS

PVS [15] je iskalna optimizacija, ki deluje zaradi implementiranega urejanja potez. PVS deluje tako, da za prvo potezo, ki je v večini primerov najboljša poteza, iz transpozicijske tabele za dano pozicijo pregleda vse nadaljnje možne poteze in po algoritmu negamax shrani to vrednost v spremenljivko alfa. V primeru, ko je bila prva poteza že pregledana, pa izvede iskanje s spremenjenim oknom [–alfa – 1, –alfa]. To iskanje izvajamo, ker želimo dobiti informacijo, ali trenutna poteza lahko zviša alfo, oziroma ali obstaja možnost, da ima poteza večjo oceno od prve pregledane poteze. Če dobimo informacijo, da ta poteza ne more povečati alfe, s pregledom te poteze zaključimo. V primeru, da je rezultat iskanja večji od trenutne najboljše poteze, izvedemo ponovno iskanje za točno oceno poteze z normalnim alfa beta oknom. Ta optimizacija deluje, ker iskanje v majhnem oknu povzroči več alfa beta rezanj, kot če poskušamo dobiti točno oceno v oknu polne alfa bete.

7.5 LMR

Optimizacija LMR [4] ponovno uporablja idejo urejanja potez. Ideja LMR je, da preiskovanje potez, za katere smo preko hevristik ocenili, da so poteze, ki najverjetneje niso najboljše v poziciji, ne poteka do polne globine, ampak globino iskanja zmanjšamo. Več potez, kot smo že preverili, manj globoko preiskujemo naslednje poteze. Seveda obstajajo pogoji, kdaj se LMR uporablja, saj bi sicer ocenjevali določene dobre poteze s prevej manjšo natančnostjo, kot bi jih morali. LMR se uporablja skupaj s prej omenjeno optimizacijo PVS. Pri oceni prve poteze se LMR nikoli ne uporablja. Pri drugih potezah se uporablja skupaj z oknom PVS [–alfa – 1, –alfa], kjer želimo dobiti informacijo, ali je trenutna poteza lahko boljša od najboljše poteze, ki smo jo

našli do sedaj. V primeru, da poteza ne more spremeniti najboljše poteze, prenehamo s preiskovanjem poteze. V primeru, da poteza lahko spremeni najboljšo možno potezo, potezo ponovno preiščemo z oknom PVS, ampak brez zmanjšane globine. Nadaljnje dogajanje ostane enako kot pri zgoraj navedeni implementaciji.

7.6 Rezanje mate distance

Še zadnja optimizacija, ki smo jo dodali programu, je rezanje mate distance [6]. Ta optimizacija se uporablja samo v primeru, ko je pri trenutnem iskanju vrednosti najboljše ali najslabše poteze mat. V naši implementaciji je ocena mata zapisana z določeno veliko vrednostjo in dodanim zamikom, ki nam pove število potez do mata. V primerih, ko je prisoten neizogiben mat, ta optimizacija preveri trenutno globino in globino neizogibnega mata. Če globine mata ne more izboljšati, preneha s preiskovanjem pozicije.

Poglavje 8

Ocenjevanje in rezultati

8.1 Vmesnik za komunikacijo

Imamo program za analiziranje šahovskih pozicij, ampak še vedno nimamo dobrega načina za komuniciranje s programom. Na tej točki je edini način za komunikacijo s programom nastavljanje pozicije v kodi pred izvajanjem programa, kar je zelo neučinkovito za uporabo, saj moramo po vsaki novi potezi ustvariti program in spremeniti pozicijo znotraj kode. Zato smo implementirali osnovni del vmesnika UCI [2], ki ga uporabljajo sodobni programi za analiziranje šahovskih pozicij in grafični uporabniški vmesniki za prikaz pozicij in možnost igranja proti programu. Vmesnik deluje s pomočjo standardnega vhoda in izhoda operacijskega sistema. Tako omogoča komunikacijo med različnimi programi s standardiziranimi sporočili, ki si jih programi medsebojno izmenjujejo. Naša implementacija trenutno podpira naslednje ukaze:

- **Uci:** ukaz uci izvede drug program, ki želi komunicirati z našim programom. Ta ukaz nam pove, da želi drugi program komunicirati po protokolu uci. Nazaj pa pričakuje nekaj osnovnih podatkov o imenu programa, avtorju in potrditvi komunikacijskega protokola.

- **Isready:** ob prejetju ukaza isready program pričakuje potrditev. Ta

ukaz da programu čas za izvedbo raznih inicializacij, ki jih potrebuje.

- **Ucinewgame:** ta ukaz ponazarja pričetek nove igre. V tem primeru ponastavimo zgodovinsko hevristiko in transpozicijske tabele na začetno stanje.

- **Position startpos moves <poteze>:** ukaz je razdeljen na dva dela. Prvi del position startpos pove našem programu, da mora ponastaviti pozicijo. Drugi del je sestavljen iz ključne besede moves in niza potez. Poteze so ločene s presledki. Poteza je predstavljena s polja, iz katerega se figura premika in polja, na katero se figura premakne. V primeru promocije kmeta je dodana še črka, ki ponazarja figuro, v katero promoviramo.

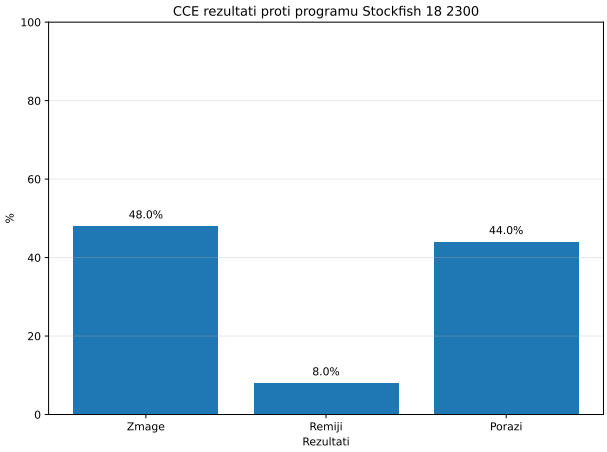
- **Go movetime <milisekunde>:** ta ukaz nam sporoči, da lahko pričnemo z analizo pozicije. Pove nam tudi, koliko časa ima naš program za izračun najboljše poteze. V naši implementaciji mora biti čas za potezo več kot 100 milisekund. Po poteku časa program vrne najboljšo potezo, zapisano enako, kot so poteze zapisane pri prejšnjem ukazu.

- **Quit:** ukaz programu sporoči, da lahko zaključi z izvajanjem.

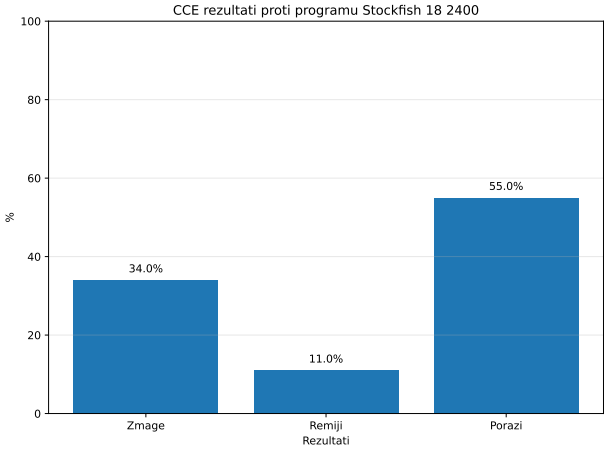
8.2 Načini ocenjevanja in rezultati

Med implementacijo smo uporabljali dva načina testiranja našega programa CCE. Prvi način je igranje sam proti sebi. Ta način nam omogoča ocenjevanje programa med iteracijami. Če nova različica programa premaga staro različico, pomeni, da smo program z dodajanjem raznih optimizacij in hevristik izboljšali. V primeru, da je prejšnja različica boljša, to pomeni, da smo novo funkcionalnost ali optimizacijo napačno implementirali, ali pa smo z dodajanjem ali spreminjanjem hevristike pokvarili pomembnost nekaterih drugih hevristik. V obeh primerih moramo preveriti, kaj je bilo izvedeno narobe. Vedeti moramo, da na osnovi le ene igre ne moremo nima določiti,

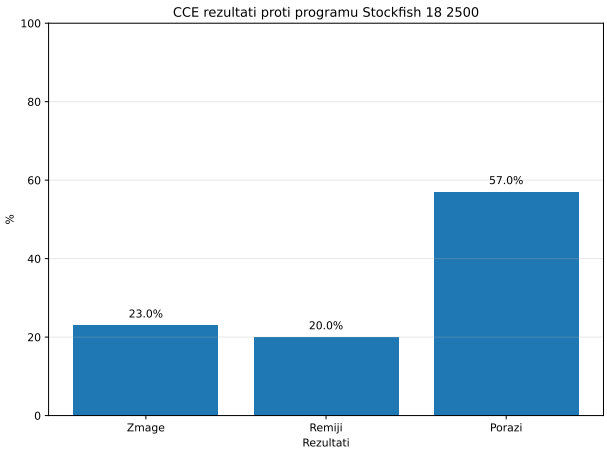
ali je program boljši ali slabši. Zato smo pri testiranju uporabili zbirko 50 otvoritev, ki vsebujejo do 10 vnaprej definiranih potez. Pri testiranju moramo biti pozorni tudi na barvo igralca pri otvoritvi, saj ne želimo igrati enake otvoritve z enako barvo. Zato smo skupaj testirali 50 otvoritev s spremembo barve za vsako, kar skupno pomeni 100 iger. Drug način testiranja je bil proti enemu izmed najboljših programov za analiziranje šahovskih pozicij Stockfish. Stockfish ima možnost omejevanja moči. Za naše teste uporabljamo program Stockfish 18 [11]. Edina sprememba pri nastavitvah programa je omejena moč UCI Elo. Čas za potezo smo omejili na eno sekundo. Testirali smo enako kot pri samotestiranju. Testiranje moči proti programu Stockfish z omejenimi močmi 2300, 2400 in 2500 si lahko pogledamo na histogramih 8.1.



(a) Histogram rezultatov igranja našega programa proti programu Stockfish 18 z omejeno močjo 2300



(b) Histogram rezultatov igranja našega programa proti programu Stockfish 18 z omejeno močjo 2400



(c) Histogram rezultatov igranja našega programa proti programu Stockfish 18 z omejeno močjo 2500

Slika 8.1: Predstavitev rezultatov

Iz zgoraj prikazanih rezultatov lahko sklenemo, da je naš program za analiziranje šahovskih pozicij primerljiv z programom Stockfish 18 z omejeno močjo 2300.

Pri rezultatih moramo upoštevati, da se navezujejo specifično na program Stockfish 18 z omejeno močjo. Rezultatov ocene moči ne moremo neposredno primerjati z drugimi načini ocenjevanja moči, kot so rangiranje FIDE in drugi načini rangiranja raznih platform, kot sta chess.com in lichess, saj se že med posameznimi platformami ocena zelo razlikuje. Lahko pa te rezultate vzamemo kot približek moči programa v človeškem svetu šaha.

Poglavje 9

Zaključek

V naši implementaciji smo predstavili izbiro strukture figur in pozicij, način generiranja potez, igranje in izničevanje potez, razne iskalne algoritme, način evaluvacije pozicije in dodatne iskalne optimizacije, ki pohitrijo delovanje programa za analiziranje šahovskih pozicij. Dodatno smo predstavili še vmesnik za komunikacijo med programi in način testiranja našega programa. Program je presegel naša pričakovanja. Ob pričetku smo si zadali cilj, da izdelamo program z močjo okoli 2000. Izdelali pa smo program za analiziranje šahovskih pozicij, ki je primerljiv z programom Stockfish 18 [11] z omejeno močjo 2300. Med testiranjem smo opazili nekaj pomankljivosti pri igri, ki bi jih bilo potrebno popraviti. Prva pomankljivost, je preveč osnovna hevristika za analiziranje varnosti kralja v poziciji. Druga večja pomankljivost je kvaliteta igranja končnic, ki se močno razlikujejo od igre otvoritev in srednje igre. Poleg teh pomanjkljivostih, bi lahko dodali še nove funkcionalnosti. Prva zaželena funkcionalnost je možnost povezave programa na spletne storitve kot so Lichess. To bi uporabnikom omogočilo lažjo interakcijo s programom. Poleg uporabniške izkušnje bi lahko dodali še dodatne iskalne optimizacije ter nekaj novih hevristik za ocenjevanje pozicij. Glede testiranja bi lahko dodali bolj statističn način testiranja, ki bi bolj natančno oceni moč programa.

Literatura

[1] Samarth Agarwal in sod. „NIRNAY: An AI Chess Engine Based on Convolutional Neural Network, Negamax and Move Ordering“. V: *2024 11th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*. 2024, str. 1–6. DOI: 10.1109/ICRITO61523.2024.10522402.

[2] Yulian Dashev, Martin Vassilev in Alexander Penev. „A Modular Teaching-Oriented Chess Engine with Validated Move Generation and Classical Search Enhancements“. V: *2025 International Conference Automatics and Informatics (ICAI)*. 2025, str. 438–443. DOI: 10.1109/ICAI67591.2025.11324630.

[3] Sacha Droste in Johannes Fürnkranz. „Learning the Piece Values for Three Chess Variants“. V: *International Journal of Computer Games Technology* 2008 (2008). DOI: 10.3233/ICG-2008-31403. URL: https://journals.sagepub.com/doi/10.3233/ICG-2008-31403 (pridobljeno 1. 9. 2026).

[4] Kunihito Hoki in Masakazu Muramatsu. „Efficiency of three forward-pruning techniques in shogi: Futility pruning, null-move pruning, and Late Move Reduction (LMR)“. V: *Entertain. Comput.* 3 (2012), str. 51–57. URL: https://api.semanticscholar.org/CorpusID:6852129.

[5] Feng-hsiung Hsu in Jon Kleinberg. *Behind Deep Blue: Building the Computer That Defeated the World Chess Champion*. 2022.

[6] *Iskalna optimizacija rezanje mate distance*. URL: https://chessprogramming.org/Mate_Distance_Pruning (pridobljeno 17. 8. 2026).

[7] Pradyumna Kannan. *Magic Move-Bitboard Generation in Computer Chess*. Teh. poročilo. 2007. URL: http://pradu.us/old/Nov27_2008/Buzz/research/magic/Bitboards.pdf (pridobljeno 1. 9. 2026).

[8] Ishika Khokhani in sod. „Unveling Chess Algorithms Using Reinforcement Learning and Traditional Chess Approaches in AI“. V: *2023 3rd Asian Conference on Innovation in Technology (ASIANCON)*. 2023, str. 1–4. DOI: 10.1109/ASIANCON58793.2023.10269900.

[9] Rekha R Nair in sod. „Improving Alpha-Beta Pruning Efficiency in Chess Engines“. V: *2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD)*. 2025, str. 1–6. DOI: 10.1109/ITIKD63574.2025.11004942.

[10] *Program Belle*. URL: https://chessprogramming.org/Belle (pridobljeno 17. 8. 2026).

[11] *Program za anlizo šaha Stockfish 18*. URL: https://stockfishchess.org/blog/2026/stockfish-18/ (pridobljeno 17. 8. 2026).

[12] *Prva pojavitev igranja šaha proti napravi*. URL: https://en.wikipedia.org/wiki/Mechanical_Turk (pridobljeno 17. 8. 2026).

[13] *Prva šahovska naprava El Ajedrecista*. URL: https://chessprogramming.org/El_Ajedrecista (pridobljeno 17. 8. 2026).

[14] *Prvi program, ki je lahko odigral celotno igro*. URL: https://chessprogramming.org/IBM_704 (pridobljeno 17. 8. 2026).

[15] Zhiyang Qi in sod. „Optimization of Connect6 based on Principal Variation Search and Transposition Tables Algorithms“. V: *2020 Chinese Control And Decision Conference (CCDC)* (2020), str. 198–203. URL: https://api.semanticscholar.org/CorpusID:221119558.

[16] Albert Lindsey Zobrist. „Ordered Minimal Perfect Hash of the Human Genome and Implications for Duplicate Finding“. V: *2009 WRI World Congress on Computer Science and Information Engineering*. Zv. 4. 2009, str. 106–111. DOI: 10.1109/CSIE.2009.1070.